

Lab2任务报告

蒲彦丞-2200012956

Task 1

该任务根据要求一步步进行即可，然而我半夜神志不清没开wireframes让我以为自己写的有问题查了好久bug...具体代码如下:

1. 更新原顶点位置

```
glm::vec3 curr_v(0);
std::size_t num = neighbors.size();

if (num == 3) {
    curr_v = prev_mesh.Positions[i] * glm::vec3(1 - 9.0f / 16.0f);
    for (auto point : neighbors) {
        curr_v += prev_mesh.Positions[point] * glm::vec3(3.0f / 16.0f);
    }
} else {
    curr_v = prev_mesh.Positions[i] * glm::vec3(0.625f);
    for (auto point : neighbors) {
        curr_v += prev_mesh.Positions[point] * glm::vec3(0.375f / num);
    }
}
curr_mesh.Positions.push_back(curr_v);
```

2. 为边生成新顶点

边界边处理:

```
glm::vec3 curr_v = (prev_mesh.Positions[e->From()] + prev_mesh.Positions[e->To()]) * glm::vec3(0.5f);
curr_mesh.Positions.push_back(curr_v);
```

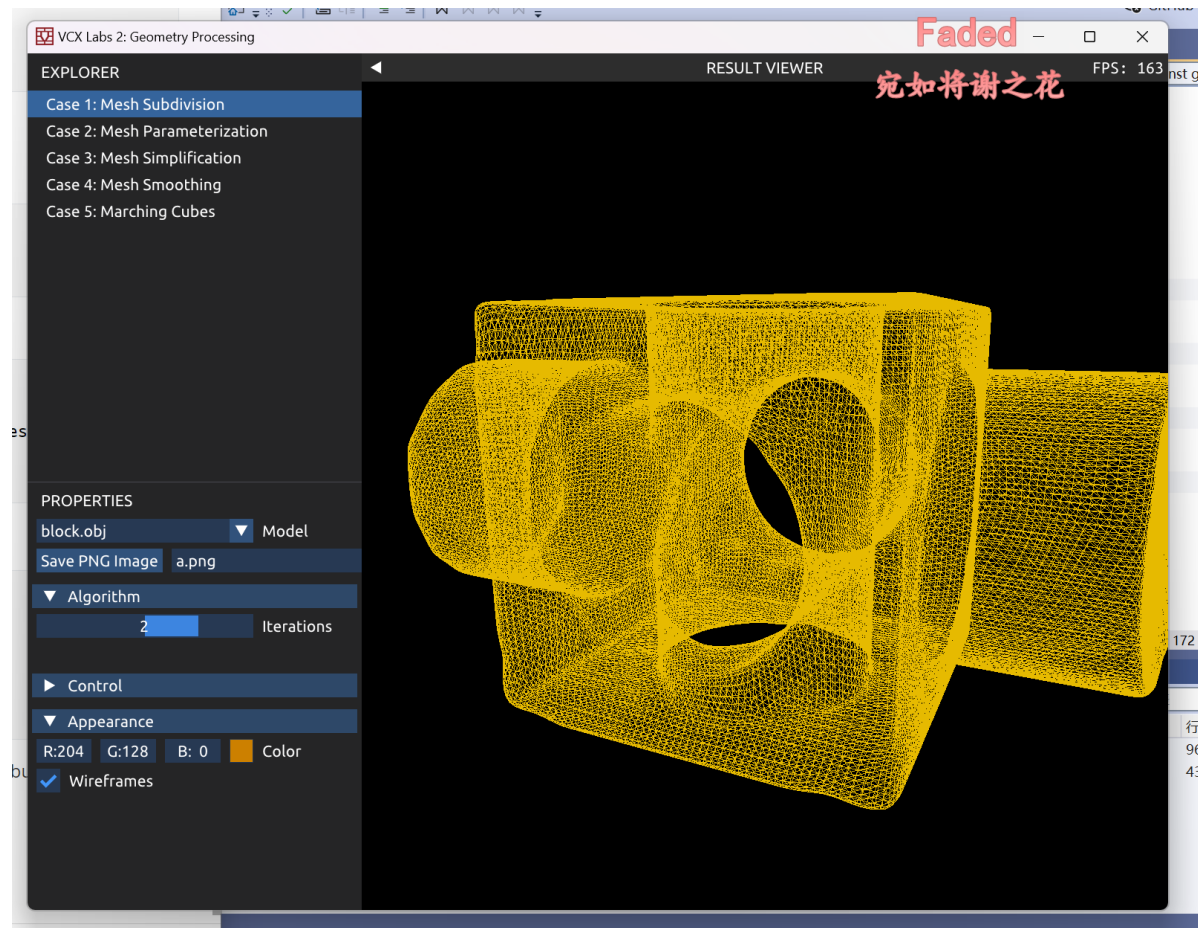
非边界边处理:

```
glm::vec3 curr_v = (prev_mesh.Positions[e->From()] + prev_mesh.Positions[e->To()]) * glm::vec3(0.375f)
    + (prev_mesh.Positions[e->OppositeVertex()] + prev_mesh.Positions[e->TwinOppositeVertex()]) * glm::vec3(0.125f);
curr_mesh.Positions.push_back(curr_v);
```

3. 重新构造面

```
std::uint32_t toInsert[4][3] = {
    {v0, m2, m1},
    {v1, m0, m2},
    {v2, m1, m0},
    {m2, m0, m1}
};
```

实现效果如下:



Task 2

这个任务中我是用了矩形映射边界，代码如下：

1. 初始化边界点的UV坐标

```
float x_max = input.Positions[0].x;
float y_max = input.Positions[0].y;
float x_min = input.Positions[0].x;
float y_min = input.Positions[0].y;

for (int i = 0; i < G.NumOfVertices(); ++i) {
    x_max = input.Positions[i].x > x_max ? input.Positions[i].x : x_max;
    x_min = input.Positions[i].x < x_min ? input.Positions[i].x : x_min;
    y_max = input.Positions[i].y > y_max ? input.Positions[i].y : y_max;
    y_min = input.Positions[i].y < y_min ? input.Positions[i].y : y_min;
}

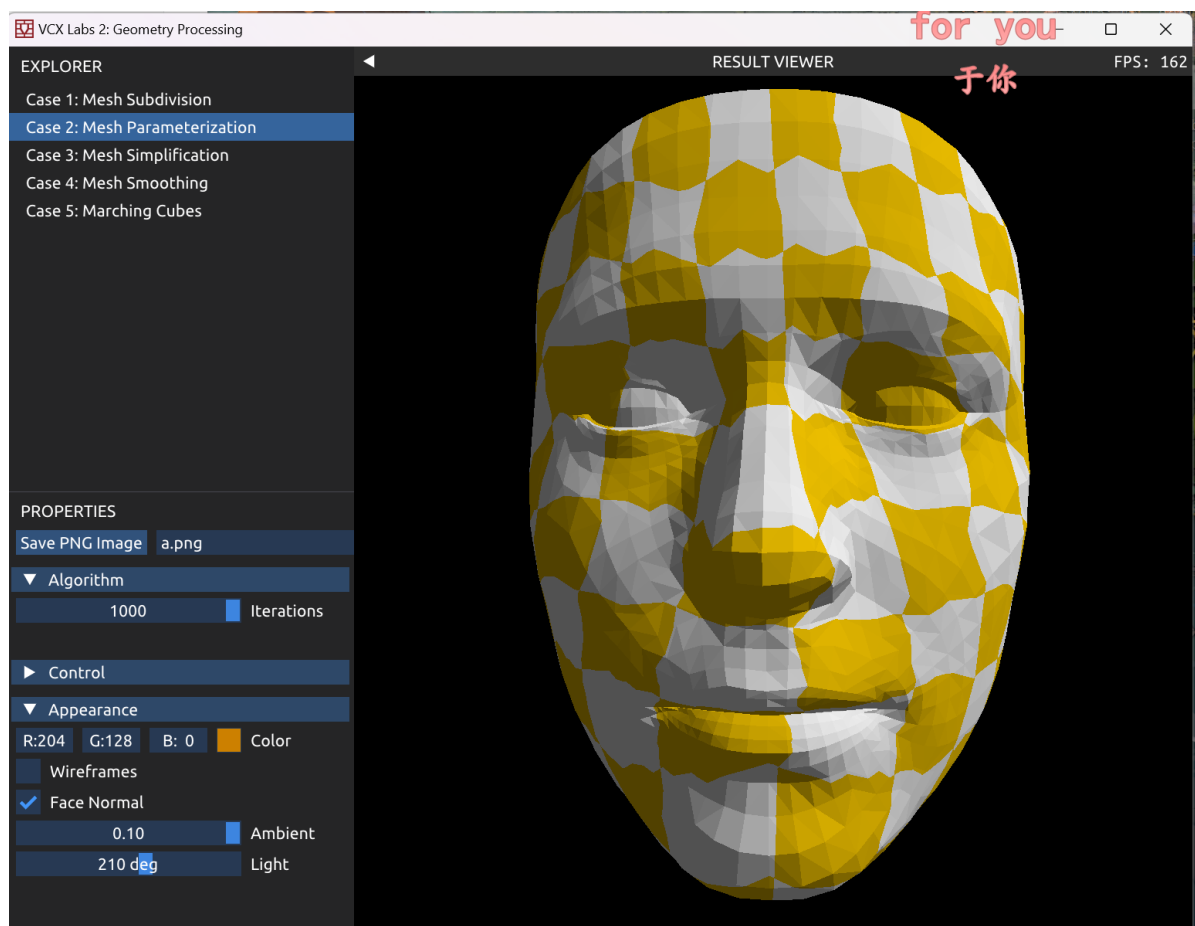
for (int i = 0; i < G.NumOfVertices(); ++i) {
    if (G.Vertex(i)->OnBoundary()) {
        output.TexCoords[i].x = (input.Positions[i].x - x_min) / (x_max -
x_min);
        output.TexCoords[i].y = (input.Positions[i].y - y_min) / (y_max -
y_min);
    }
}
```

2. Gauss-Seidel迭代求解UV坐标

```
for (int k = 0; k < numIterations; ++k) {
    for (int i = 0; i < G.NumOfVertices(); ++i) {
        if (G.Vertex(i)->OnBoundary())
            continue;

        auto neighbors = G.Vertex(i)->Neighbors();
        auto num       = neighbors.size();
        float x_new     = 0;
        float y_new     = 0;
        for (int j = 0; j < num; ++j) {
            x_new += output.TexCoords[neighbors[j]].x;
            y_new += output.TexCoords[neighbors[j]].y;
        }
        output.TexCoords[i].x = (float) x_new / num;
        output.TexCoords[i].y = (float) y_new / num;
    }
}
```

实现效果如下:



Task 3

助教老师们确实很有经验，我搞错矩阵下标想了好久，看到tips才改过来，代码如下：

1. 为每个初始顶点计算二次代价矩阵Kp

```
auto UpdateQ {
    [&G, &output] (DCEL::Triangle const * f) -> glm::mat4 {
        glm::mat4 Kp;
        glm::vec3 v0 = output.Positions[f->VertexIndex(0)];
        glm::vec3 v1 = output.Positions[f->VertexIndex(1)];
        glm::vec3 v2 = output.Positions[f->VertexIndex(2)];

        glm::vec3 e1 = v1 - v0;
        glm::vec3 e2 = v2 - v0;
        glm::vec3 n = glm::normalize(glm::cross(e1, e2));
        float d = 0.0f - glm::dot(v0, n);
        glm::vec4 plane { n, d };
        Kp = glm::outerProduct(plane, plane);

        return Kp;
    }
};
```

2. 根据顶点对创建收缩对

```
static constexpr auto MakePair {
    [] (DCEL::HalfEdge const * edge,
        glm::vec3 const & p1,
        glm::vec3 const & p2,
        glm::mat4 const & Q
    ) -> ContractionPair {
        glm::mat4 P = Q;
        P[0][3] = 0.0f;
        P[1][3] = 0.0f;
        P[2][3] = 0.0f;
        P[3][3] = 1.0f;
        glm::vec4 v(0);
        glm::vec4 v1 = { p1, 1.0f };
        glm::vec4 v2 = { p2, 1.0f };

        if (glm::determinant(P) < 0.001) {
            v = (v1 + v2) / 2.0f;
        } else {
            v = glm::inverse(P)[3];
        }

        glm::vec4 tmp = Q * v;
        float cost = glm::dot(v, tmp);

        return { edge, v, cost };
    }
};
```

3. 更新顶点的二次代价矩阵和收缩对

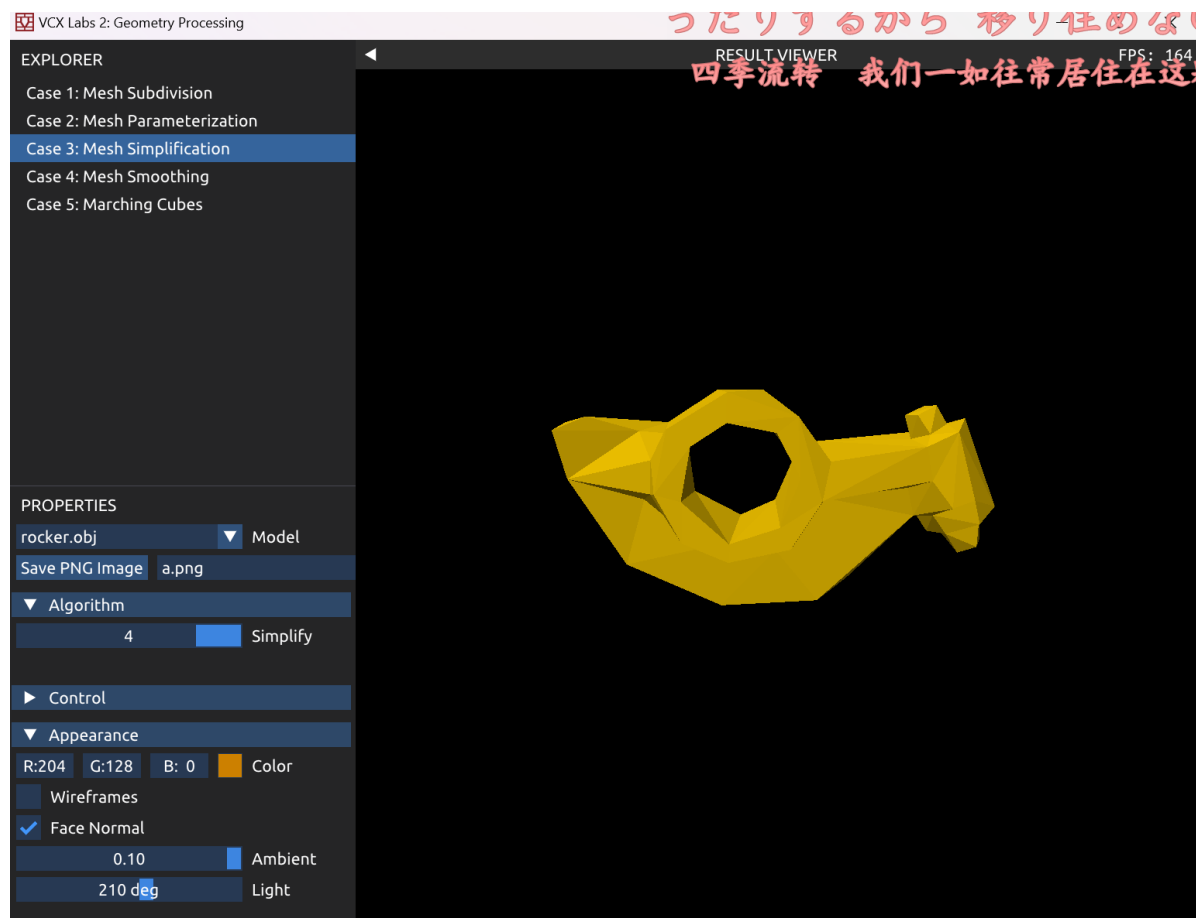
```
for (auto e : ring) {
    auto tf = e->Face();
    glm::mat4 nQ = updateQ(tf);

    auto delta = nQ - Kf[G.Indexof(tf)];
    Qv[e->From()] += delta;
    Qv[e->To()] += delta;

    Qv[v1] += nQ;
    Kf[G.Indexof(tf)] = nQ;
}

for (auto e : ring) {
    pairs[pair_map[G.Indexof(e->NextEdge())]] = MakePair(
        e->NextEdge(), input.Positions[e->NextEdge()->From()],
        input.Positions[e->NextEdge()->To()],
        Qv[e->NextEdge()->From()] + Qv[e->NextEdge()->To()]
    );
}
```

实现结果如图:



Task 4

对余切的极值情况我简单的归零处理了，这让我的结果没有那么尖锐

1.计算余切值的函数:

```
static constexpr auto GetCotangent {
    [] (glm::vec3 vAngle, glm::vec3 v1, glm::vec3 v2) -> float {
        glm::vec e1 = v1 - vAngle;
        glm::vec e2 = v2 - vAngle;

        double dot = glm::dot(e1, e2);
        double cross = glm::length(glm::cross(e1, e2));

        if (cross < 1e-4 || dot * cross < 0)
            return 0.0f;
        else return (dot / cross);
    }
};
```

2.根据所选的 Laplacian 类型 (均匀或余切) 进行位置更新:

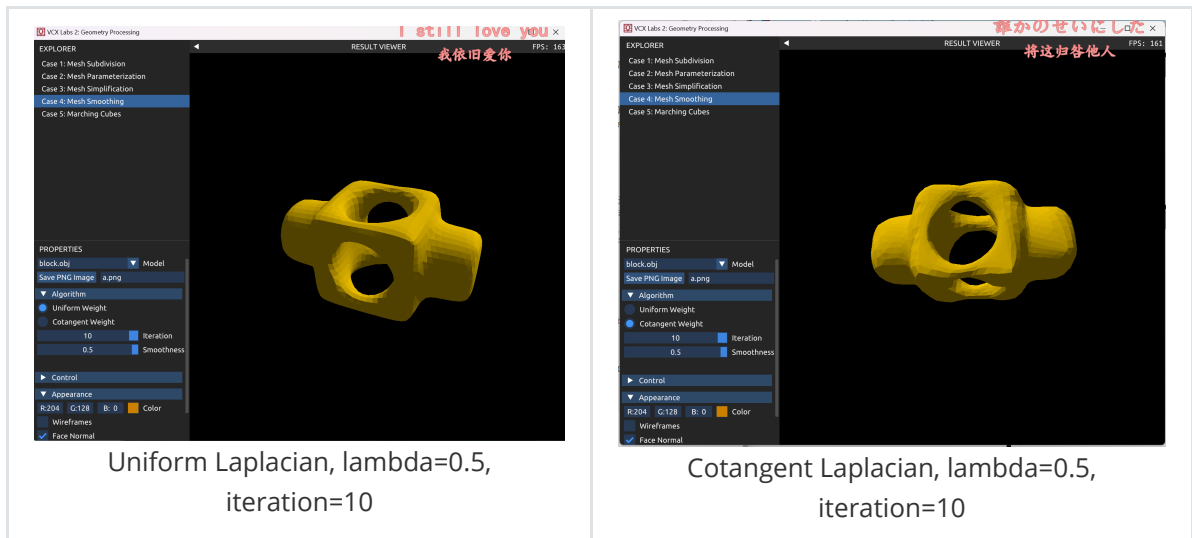
```
if (useUniformweight) {
    for (int j = 0; j < num; ++j) {
        x_new += prev_mesh.Positions[neighbors[j]].x;
        y_new += prev_mesh.Positions[neighbors[j]].y;
        z_new += prev_mesh.Positions[neighbors[j]].z;
    }

    curr_mesh.Positions[i].x = (float) ((1 - lambda) *
prev_mesh.Positions[i].x + lambda * x_new / num);
    curr_mesh.Positions[i].y = (float) ((1 - lambda) * prev_mesh.Positions[i].y
+ lambda * y_new / num);
    curr_mesh.Positions[i].z = (float) ((1 - lambda) * prev_mesh.Positions[i].z
+ lambda * z_new / num);
} else {
    float cot_sum = 0;
    for (int j = 0; j < num; ++j) {
        float cot = GetCotangent(prev_mesh.Positions[i],
prev_mesh.Positions[neighbors[(j - 1) % num]],
prev_mesh.Positions[neighbors[j]]) +
            GetCotangent(prev_mesh.Positions[i], prev_mesh.Positions[(j
+ 1) % num], prev_mesh.Positions[neighbors[j]]);
        x_new += prev_mesh.Positions[neighbors[j]].x * cot;
        y_new += prev_mesh.Positions[neighbors[j]].y * cot;
        z_new += prev_mesh.Positions[neighbors[j]].z * cot;
        cot_sum += cot;
    }

    curr_mesh.Positions[i].x = (float) ((1 - lambda) * prev_mesh.Positions[i].x
+ lambda * x_new / cot_sum);
    curr_mesh.Positions[i].y = (float) ((1 - lambda) * prev_mesh.Positions[i].y
+ lambda * y_new / cot_sum);
}
```

```
curr_mesh.Positions[i].z = (float) ((1 - lambda) * prev_mesh.Positions[i].z  
+ lambda * z_new / cot_sum);  
}
```

实现结果如下:



Task 5

应该还有一点可以改进的地方比如边界情况以及重复计算什么的，但是没时间了就这样吧

代码：

```
void MarchingCubes(Engine::SurfaceMesh & output, const std::function<float(const glm::vec3 &)> & sdf, const glm::vec3 & grid_min, const float dx, const int n) {
    // your code here:
    auto unit {
        [](int n) -> glm::vec3 {
            switch (n) {
            case 0:
                return glm::vec3(1, 0, 0);
                break;
            case 1:
                return glm::vec3(0, 1, 0);
                break;
            case 2:
                return glm::vec3(0, 0, 1);
                break;
            default: return glm::vec3(1, 0, 0);
            }
        }
    };
    std::vector<glm::vec3> edge(3 * (n + 1) * (n + 1) * (n + 1), glm::vec3(0));
    std::vector<int> edgeforidx(3 * (n + 1) * (n + 1) * (n + 1), -1);
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            for (int k = 0; k < n; ++k) {
                glm::vec3 basePoint = grid_min + dx * glm::vec3(k,
j, i);

                std::vector<glm::vec3> cube = {
                    basePoint,
                    basePoint + glm::vec3(dx, 0, 0),
                    basePoint + glm::vec3(0, dx, 0),
                    basePoint + glm::vec3(dx, dx, 0),
                    basePoint + glm::vec3(0, 0, dx),
                    basePoint + glm::vec3(dx, 0, dx),
                    basePoint + glm::vec3(0, dx, dx),
                    basePoint + glm::vec3(dx, dx, dx)
                };
                int baseEdgeidx = i * 3 * (n + 1) * (n + 1) + j * 3
* (n + 1) + 3 * k;
                std::vector<int> edgeMap = { baseEdgeidx,
                    baseEdgeidx + 3 * (n + 1),
                    baseEdgeidx + 3 * (n + 1) * (n
+ 1),
                    baseEdgeidx + 3 * (n + 1) * (n
+ 2),
                    baseEdgeidx + 1,
                    baseEdgeidx + 1 + 3 * (n + 1) *
(n + 1),
                    baseEdgeidx + 4,
```

```

(n + 1),
baseEdgeidx + 4 + 3 * (n + 1) *
baseEdgeidx + 2,
baseEdgeidx + 5,
baseEdgeidx + 2 + 3 * (n + 1),
baseEdgeidx + 3 * (n + 1) + 5
};

int v = 0;
for (int i = 0; i < 8; ++i) {
    if (sdf(cube[i]) > 0) {
        v += (1 << i);
    }
}
unsigned int edgeRes = c_EdgeStateTable[v & 0xff];
for (int j = 0; j < 12; ++j) {
    if (1 & (edgeRes >> j)) {
        if (edgeforidx[edgeMap[j]] == -1) {
            glm::vec3 src = cube[0] + dx * (j & 1) *
unit(((j >> 2) + 1) % 3) + dx * ((j >> 1) & 1) * unit(((j >> 2) + 2) % 3);
            glm::vec3 dst = src + dx * unit(j >> 2);
            glm::vec3 point = (sdf(dst) / (sdf(dst) -
sdf(src))) * src + (-sdf(src) / (sdf(dst) - sdf(src))) * dst;
            edgeforidx[edgeMap[j]] = output.Positions.size();
            output.Positions.push_back(point);
        }
    }
}
int ptr = 0;
while (c_EdgeOrdsTable[v][ptr] != -1) {

output.Indices.push_back(edgeforidx[edgeMap[c_EdgeOrdsTable[v][ptr]]]);

output.Indices.push_back(edgeforidx[edgeMap[c_EdgeOrdsTable[v][ptr + 1]]]);

output.Indices.push_back(edgeforidx[edgeMap[c_EdgeOrdsTable[v][ptr + 2]]]);

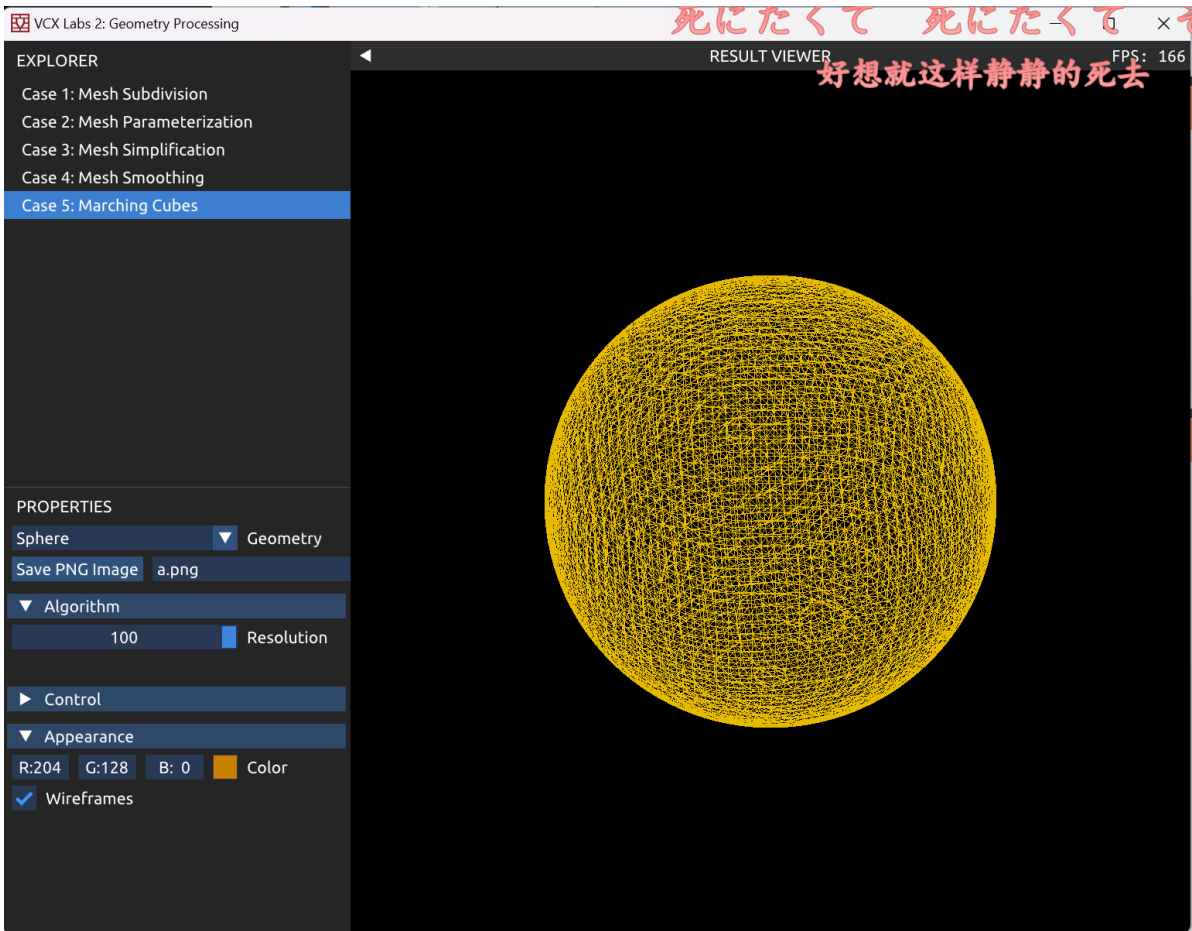
output.Indices.push_back(edgeforidx[edgeMap[c_EdgeOrdsTable[v][ptr + 2]]]);

output.Indices.push_back(edgeforidx[edgeMap[c_EdgeOrdsTable[v][ptr + 1]]]);

output.Indices.push_back(edgeforidx[edgeMap[c_EdgeOrdsTable[v][ptr]]]);
        ptr += 3;
    }
}
}
}
}
}

```

实验结果如下:



总结:

这次算法确实要难一些，我做第三题时还是看了蛮久才想明白simplify的代码逻辑的，还有就是半边结构也不熟悉，做起来出现了很多困难.